

Foundations of Language Modeling

Chapter 6: Constructing the Transformer Block

1 The Position-wise Feed-Forward Network

While the Multi-Head Attention mechanism allows tokens to communicate and aggregate information across the sequence, it consists entirely of linear projections and weighted sums. To introduce deep, non-linear representational power, the Transformer employs a Feed-Forward Network (FFN) applied to each token position independently.

Definition 1.1 (Position-wise Feed-Forward Network). *Let $\mathbf{x}_t \in \mathbb{R}^C$ be the embedding of the token at step t after the attention mechanism. The FFN consists of two linear transformations with a non-linear activation function (typically ReLU or GELU) in between:*

$$FFN(\mathbf{x}_t) = \max(0, \mathbf{x}_t \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2 \quad (1)$$

By convention in the standard Transformer architecture, the inner hidden layer expands the dimensionality by a factor of 4 (i.e., $\mathbf{W}_1 \in \mathbb{R}^{C \times 4C}$ and $\mathbf{W}_2 \in \mathbb{R}^{4C \times C}$).

Intuition: If Self-Attention is the "communication" phase where tokens gather context from each other, the Feed-Forward Network is the "computation" phase where each token individually processes the newly gathered information.

2 Layer Normalization

Deep neural networks often suffer from unstable gradients and internal covariate shift. To stabilize training, the Transformer utilizes **Layer Normalization** (LayerNorm).

Definition 2.1 (Layer Normalization). *Given a feature vector $\mathbf{x} \in \mathbb{R}^C$ for a specific token in a specific batch, LayerNorm normalizes the vector across the channel dimension C . We compute the empirical mean μ and variance σ^2 :*

$$\mu = \frac{1}{C} \sum_{j=1}^C x_j, \quad \sigma^2 = \frac{1}{C} \sum_{j=1}^C (x_j - \mu)^2 \quad (2)$$

The normalized vector $\hat{\mathbf{x}}$ is then scaled and shifted by learnable parameters $\gamma, \beta \in \mathbb{R}^C$:

$$LayerNorm(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (3)$$

where ϵ is a small constant for numerical stability, and $\odot$ denotes element-wise multiplication.

Remark 2.2 (LayerNorm vs. BatchNorm). *Unlike Batch Normalization, which normalizes across the batch dimension for a specific feature, Layer Normalization normalizes across the feature (channel) dimension for a specific token. This ensures the normalization is entirely independent of the batch size and the sequence length, which is crucial for autoregressive language models.*

3 Residual Connections and the Pre-Norm Architecture

As Transformers become deeper (stacking many blocks), the gradients computed during backpropagation can vanish or explode. To guarantee a "gradient superhighway" directly from the target back to the input, we use **Residual Connections** (also called skip connections).

Definition 3.1 (Residual Connection). *Let $f(\mathbf{x})$ represent a non-trivial transformation (like Multi-Head Attention or the FFN). A residual connection adds the original input back to the output of the transformation:*

$$\mathbf{y} = \mathbf{x} + f(\mathbf{x}) \tag{4}$$

In the original "Attention Is All You Need" paper, LayerNorm was applied *after* the residual addition (Post-Norm). However, modern implementations (including GPT-2 and GPT-3) use the **Pre-Norm** formulation, applying LayerNorm *before* the transformation, which significantly improves training stability.

Definition 3.2 (The Transformer Block). *A complete Transformer Decoder Block applies Pre-Norm Self-Attention followed by a Pre-Norm Feed-Forward Network, both wrapped in residual connections:*

$$\mathbf{x}' = \mathbf{x} + \text{MultiHeadAttention}(\text{LayerNorm}(\mathbf{x})) \tag{5}$$

$$\mathbf{x}'' = \mathbf{x}' + \text{FFN}(\text{LayerNorm}(\mathbf{x}')) \tag{6}$$

4 Worked Examples and Solved Exercises

4.1 Mathematical Exercise: Layer Normalization by Hand

Problem: Consider a simplified scenario where a token's embedding dimension is $C = 3$. The raw vector before normalization is $\mathbf{x} = [2, 4, 6]^\top$. Assuming the learnable parameters are initialized to $\gamma = [1, 1, 1]^\top$ and $\beta = [0, 0, 0]^\top$, and ignoring ϵ , compute the output of the LayerNorm operation.

Solution: Step 1: Compute the mean μ .

$$\mu = \frac{1}{3}(2 + 4 + 6) = \frac{12}{3} = 4$$

Step 2: Compute the variance σ^2 .

$$\begin{aligned}\sigma^2 &= \frac{1}{3} [(2 - 4)^2 + (4 - 4)^2 + (6 - 4)^2] \\ &= \frac{1}{3} [(-2)^2 + 0^2 + 2^2] \\ &= \frac{1}{3} [4 + 0 + 4] = \frac{8}{3} \approx 2.667\end{aligned}$$

Step 3: Normalize the vector. We divide the mean-shifted vector by the standard deviation $\sqrt{8/3} \approx 1.633$.

$$\begin{aligned}\hat{x}_1 &= \frac{2 - 4}{1.633} \approx -1.225 \\ \hat{x}_2 &= \frac{4 - 4}{1.633} = 0 \\ \hat{x}_3 &= \frac{6 - 4}{1.633} \approx 1.225\end{aligned}$$

Because γ is a vector of ones and β is a vector of zeros, the final output is $\hat{\mathbf{x}} = [-1.225, 0, 1.225]^\top$. Notice that the resulting vector has exactly a mean of 0 and a variance of 1.

4.2 Coding Example: The Feed-Forward Network

Problem: Implement the position-wise FeedForward module in PyTorch. Include the $4\times$ expansion of the hidden dimension, a ReLU activation, and a Dropout layer for regularization.

Solution:

```
1 import torch.nn as nn
2
3 class FeedForward(nn.Module):
4     """ A simple linear layer followed by a non-linearity """
5     def __init__(self, n_embd, dropout):
6         super().__init__()
7         self.net = nn.Sequential(
8             # Inner layer expands dimensionality by 4
9             nn.Linear(n_embd, 4 * n_embd),
10            nn.ReLU(),
11            # Projection layer brings it back to the residual pathway dimension
12            nn.Linear(4 * n_embd, n_embd),
13            # Regularization
14            nn.Dropout(dropout)
15        )
```

```

16
17     def forward(self, x):
18     return self.net(x)
19

```

4.3 Coding Exercise: The Complete Transformer Block

Problem: Using the previously defined `MultiHeadAttention` and `FeedForward` modules, construct the complete `Block` module representing a single layer of the GPT architecture. Implement the **Pre-Norm** architecture and include residual connections.

Solution:

```

1     class Block(nn.Module):
2         """ Transformer block: communication followed by computation """
3         def __init__(self, n_embd, n_head, block_size, dropout):
4             # n_embd: embedding dimension, n_head: the number of heads we'd like
5             super().__init__()
6             head_size = n_embd // n_head
7
8             # Communication phase
9             self.sa = MultiHeadAttention(n_head, head_size, n_embd, block_size)
10
11            # Computation phase
12            self.ffwd = FeedForward(n_embd, dropout)
13
14            # Layer Normalizations
15            self.ln1 = nn.LayerNorm(n_embd)
16            self.ln2 = nn.LayerNorm(n_embd)
17
18            def forward(self, x):
19                # Pre-norm formulation: LayerNorm is applied *before* the transformation.
20                # The residual connection x + ... ensures gradients flow easily.
21
22                # 1. Communication over the sequence
23                x = x + self.sa(self.ln1(x))
24
25                # 2. Computation for each token independently
26                x = x + self.ffwd(self.ln2(x))
27
28            return x
29

```